WAREHOUSE INVENTORY SYSTEM – PRODUCT LOCATOR

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

By

V SAMEER REDDY	**2520030276**
CH SAHASRA GAYATHRI	**2520030306**
VENNELA EDALA	**2520030288**

Under the Esteemed Guidance of

Dr. V. Muniraju Naidu
Assistant Professor
Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

Declaration

We hereby declare that the project report entitled **"Warehouse Inventory System – Product Locator"** is a record of bona fide work carried out by us in partial fulfillment of the requirements for the award of the Bachelor of Technology degree in Computer Science and Engineering.

This project focuses on the application of Data Structures and Algorithms to develop a product-searching system for warehouse inventory. The implementation uses Java programming and incorporates algorithms such as Knuth-Morris-Pratt (KMP), HashMap-based searching, and Levenshtein Distance for fuzzy matching.

We declare that the work presented in this report has been prepared for academic purposes and that the sources used have been appropriately acknowledged.

V SAMEER REDDY **2520030276**
CH SAHASRA GAYATHRI **2520030306**
VENNELA EDALA **2520030288**

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING



CERTIFICATE

This is to certify that the project report entitled **"Warehouse Inventory System – Product Locator"** is a bona fide work carried out by **Ch. Sahasra Gayathri (25200303036), V. Sameer Reddy (2520030276), and Vennela Edala (252030288)** in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering during the academic year 2026–2027.

The project demonstrates the application of Data Structures and Algorithms to product identification, searching, and locating within a warehouse inventory dataset.

The work presented in this report is submitted for academic evaluation.

Signature of the Guide

Signature of the Course Coordinator **Signature of the HOD**

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to the Department of Computer Science and Engineering, KLH Bachupally, for providing us with the opportunity to undertake this Data Structures and Algorithms project entitled “Warehouse Inventory System – Product Locator.”

We express our sincere thanks to our project guide, Dr. S. Vinay Kumar, Associate Professor, Department of Computer Science and Engineering, for his valuable guidance and support throughout the development of this project. His guidance helped us understand the practical application of data structures and algorithms in solving real-world problems.

We also extend our gratitude to Dr. P. Venkateshwara Rao, Associate Professor and Head of the Department of Computer Science and Engineering, for providing the necessary academic support and encouragement.

Finally, we thank everyone who contributed directly or indirectly to the successful completion of this project. Working on this project has helped us strengthen our understanding of Java programming, algorithm design, file handling, and problem-solving.

Abstract

Warehouse inventory management involves organizing and locating products efficiently. In warehouses containing a large number of products distributed across multiple categories, identifying a particular item can become time-consuming when searching is performed manually or through inefficient techniques. An effective product locator can reduce the effort required to identify products and determine their storage locations.

The project titled **"Warehouse Inventory System – Product Locator"** is a Java-based application developed to demonstrate the practical use of Data Structures and Algorithms in product searching. The system maintains product information such as product ID, product name, storage location, and source category file. Product information is organized using text files, with separate files representing different product categories.

The application provides a menu-driven interface through which users can search for products using different searching techniques. The Knuth-Morris-Pratt (KMP) algorithm is used for efficient pattern matching in product names. A HashMap is used to support product identification through product IDs. The Levenshtein Distance algorithm, based on dynamic programming, supports fuzzy searching by identifying product names that are similar to the user's input, helping handle spelling mistakes and typing errors.

The main objective of this project is to demonstrate how suitable data structures and algorithms can improve product-searching operations. The system combines exact identification, substring matching, and approximate text matching in one application. The project also provides practical exposure to algorithm implementation, file handling, modular programming, and performance analysis.

The proposed system can serve as a foundation for a more comprehensive warehouse management application. Future improvements may include a graphical user interface, database integration, stock quantity management, barcode scanning, and real-time inventory updates.

Keywords: Data Structures and Algorithms, Warehouse Inventory, Java, KMP Algorithm, HashMap, Levenshtein Distance, Dynamic Programming, Product Search.

TABLE OF CONTENTS

1. Introduction
   - 1.1 Overview
   - 1.2 Problem Statement
   - 1.3 Objectives
   - 1.4 Scope of the Project
   - 1.5 Existing System
   - 1.6 Proposed System

2. System Requirements
   - 2.1 Functional Requirements
   - 2.2 Non-Functional Requirements
   - 2.3 Hardware Requirements
   - 2.4 Software Requirements
   - 2.5 Data Requirements

3. System Design
   - 3.1 System Architecture
   - 3.2 System Workflow
   - 3.3 Data Structures Used
   - 3.4 Algorithms Used
   - 3.5 Module Description
   - 3.6 Complexity Analysis

4. Implementation
   - 4.1 Implementation Overview
   - 4.2 Product Data Representation
   - 4.3 Dataset Loading
   - 4.4 KMP-Based Product Search
   - 4.5 HashMap-Based Product Identification
   - 4.6 Fuzzy Search Using Levenshtein Distance
   - 4.7 Main Menu and Control Flow
   - 4.8 Pseudocode

5. Results and Discussion
   - 5.1 System Output
   - 5.2 Functional Testing
   - 5.3 Algorithm Analysis
   - 5.4 Result Discussion

6. Challenges and Limitations
   - 6.1 Challenges Faced
   - 6.2 Solutions Applied
   - 6.3 Limitations of the System

7. Conclusion and Future Work
   - 7.1 Conclusion
   - 7.2 Future Enhancements
   - 7.3 Skills Learned

References

CHAPTER 1: INTRODUCTION

1.1 Overview

A warehouse is a facility used to store, organize, and manage products before they are distributed, sold, or transferred to other locations. Efficient product identification is an important part of warehouse operations because employees often need to locate specific items among numerous products stored in different sections.

As the number of products increases, searching for a particular item becomes more challenging. A product may be identified by its unique identification number or by its name. Searching by name may also require handling partial words, differences in letter case, and spelling mistakes.

Data Structures and Algorithms provide techniques for organizing information and retrieving it efficiently. Data structures determine how information is stored, while algorithms define the steps used to process and search that information.

The **Warehouse Inventory System – Product Locator** is developed using Java to demonstrate the practical application of these concepts. The application stores product information in category-based text files and provides a menu-driven interface for searching products.

The system uses three primary techniques:

- **Knuth-Morris-Pratt (KMP) algorithm:** Searches for a character pattern within a product name.
- **HashMap:** Supports efficient product lookup using a product ID as the key.
- **Levenshtein Distance:** Identifies product names that are similar to the entered search term, even when the input contains typing errors.

By combining these techniques, the application demonstrates how different algorithms can address different search requirements within the same system.

1.2 Problem Statement

In a warehouse containing many products, locating a specific item manually can consume considerable time and effort. When product information is stored across multiple category files, finding the correct product and its storage location becomes more difficult.

Basic searching methods may not efficiently support all types of searches. Searching by product name requires text matching, while searching by product ID requires a mechanism for identifying the corresponding record. Furthermore, users may enter incorrect spellings or incomplete product names, causing ordinary exact-match searches to fail.

Therefore, there is a need for a product locator that organizes product information and provides multiple searching techniques to identify products accurately and efficiently.

The problem addressed by this project is the development of a Java-based application that searches a warehouse inventory dataset using product IDs, product-name patterns, and approximate name matching.

1.3 Objectives

The main objectives of the project are:

1. To develop a menu-driven warehouse product locator using Java.
2. To organize product information into category-based text files.
3. To implement the Knuth-Morris-Pratt algorithm for pattern matching in product names.
4. To use a HashMap for product identification through unique product IDs.
5. To implement Levenshtein Distance using dynamic programming for fuzzy product searching.
6. To display relevant product information, including product ID, product name, and storage location.
7. To handle invalid input and cases in which a product cannot be found.
8. To understand the practical use of data structures and algorithms in a real-world application.

1.4 Scope of the Project

The scope of this project is limited to searching and locating products within a predefined warehouse inventory dataset.

The application includes the following functionalities:

- Loading product information from category-based text files.
- Representing products using their IDs, names, and locations.
- Searching for products by name using KMP-based pattern matching.
- Searching for products by ID using a HashMap.
- Performing fuzzy name matching using Levenshtein Distance.
- Displaying matching product information.
- Handling unsuccessful searches and invalid menu selections.

The system is designed primarily as an educational project to demonstrate algorithmic concepts. It does not necessarily include advanced inventory operations such as purchasing, billing, user authentication, live stock updates, or automatic warehouse routing.

1.5 Existing System

In a basic warehouse environment, product information may be maintained in registers, spreadsheets, or separate files. Employees may need to search manually through these records to identify a product and determine its location.

Traditional linear searching examines records one by one until a match is found. Although this method is simple, its performance may become less efficient as the number of records increases.

Basic exact-name searching also has limitations. It may fail when the user enters only part of a product name or makes a spelling mistake. In addition, searching across multiple files can increase the effort required to retrieve information.

The main limitations of a basic system are:

- Time-consuming manual searching.
- Difficulty locating products across multiple categories.
- Limited support for partial text matching.
- Failure to identify similar names when spelling errors occur.
- Lack of a unified search interface.

1.6 Proposed System

The proposed system is a Java-based warehouse product locator that uses suitable data structures and algorithms to improve product identification.

The application loads product records from category-based text files and makes them available through a menu-driven interface.

KMP is used for searching patterns in product names without repeatedly comparing characters that have already been matched. A HashMap supports product-ID lookup through key-based access. Levenshtein Distance calculates the minimum number of character insertions, deletions, and substitutions required to transform one string into another.

These techniques address different search requirements. KMP supports pattern matching, HashMap supports identifier-based lookup, and Levenshtein Distance supports approximate text matching.

The proposed system provides a practical demonstration of algorithm selection and data organization in a warehouse-related problem.

CHAPTER 2: SYSTEM REQUIREMENTS

2.1 Functional Requirements

Functional requirements describe the operations that the application must perform.

FR1: Dataset Loading

The system must read product information from the available category-based text files.

FR2: Product Representation

The system must represent product information using fields such as product ID, product name, storage location, and source file.

FR3: Product Search by Name

The system must allow users to enter a search pattern and identify product names containing that pattern using the KMP algorithm.

FR4: Product Search by ID

The system must allow users to search for a product using its unique product ID through a HashMap.

FR5: Fuzzy Product Search

The system must compare the entered search term with product names using Levenshtein Distance and identify sufficiently similar names.

FR6: Display Search Results

The system must display the available information of matching products, including their IDs, names, and locations.

FR7: Handle Unsuccessful Searches

The system must display an appropriate message when no matching product is found.

FR8: Menu Navigation

The system must allow users to select a search method and exit the application through a menu-driven interface.

FR9: Input Handling

The system should handle case differences and invalid menu selections appropriately.

2.2 Non-Functional Requirements

Performance: The system should search product records efficiently using appropriate algorithms and data structures.

Usability: The menu-driven interface should be simple and understandable.

Reliability: The system should handle unsuccessful searches and common input errors without terminating unexpectedly.

Maintainability: The program should be organized into separate methods or modules to simplify testing and modification.

Portability: Since the application is developed in Java, it can run on systems with a compatible Java Development Kit.

Scalability: The design should allow additional product categories and records to be incorporated, subject to available memory and implementation constraints.

Correctness: The search algorithms should return results consistent with their defined matching rules.

2.3 Hardware Requirements

The following are suitable basic hardware requirements for developing and running the application.

- Processor: Intel or equivalent processor.
- RAM: 4 GB or higher recommended.
- Storage: Sufficient space for Java tools, source code, and product data files.
- Input devices: Keyboard and mouse.
- Output device: Monitor or display.

These are general recommended specifications rather than experimentally determined minimum requirements.

2.4 Software Requirements

- **Programming Language:** Java.
- **Java Environment:** A compatible JDK.
- **Development Environment:** Any Java-compatible IDE or text editor.
- **Data Storage:** Category-based text files.
- **Interface:** Console-based menu.
- **Operating System:** Windows, Linux, or macOS with a compatible Java environment.

2.5 Data Requirements

The application uses product records stored in category-based text files within the project's data directory.

Each product record contains information such as:

Field	Description
Product ID	Unique identifier for the product
Product Name	Name of the product
Storage Location	Warehouse location associated with the product
Source File	Category file from which the record was loaded

The project is organized into directories such as `src`, `data`, `docs`, `results`, and `reports`, separating source code, input data, documentation, and output materials.

The application can load records from multiple category files. The number of records available to the program depends on the actual contents of those files.

CHAPTER 3: SYSTEM DESIGN

3.1 System Architecture

The system follows a simple layered workflow consisting of data storage, data loading, search processing, and result presentation.

Architecture Diagram

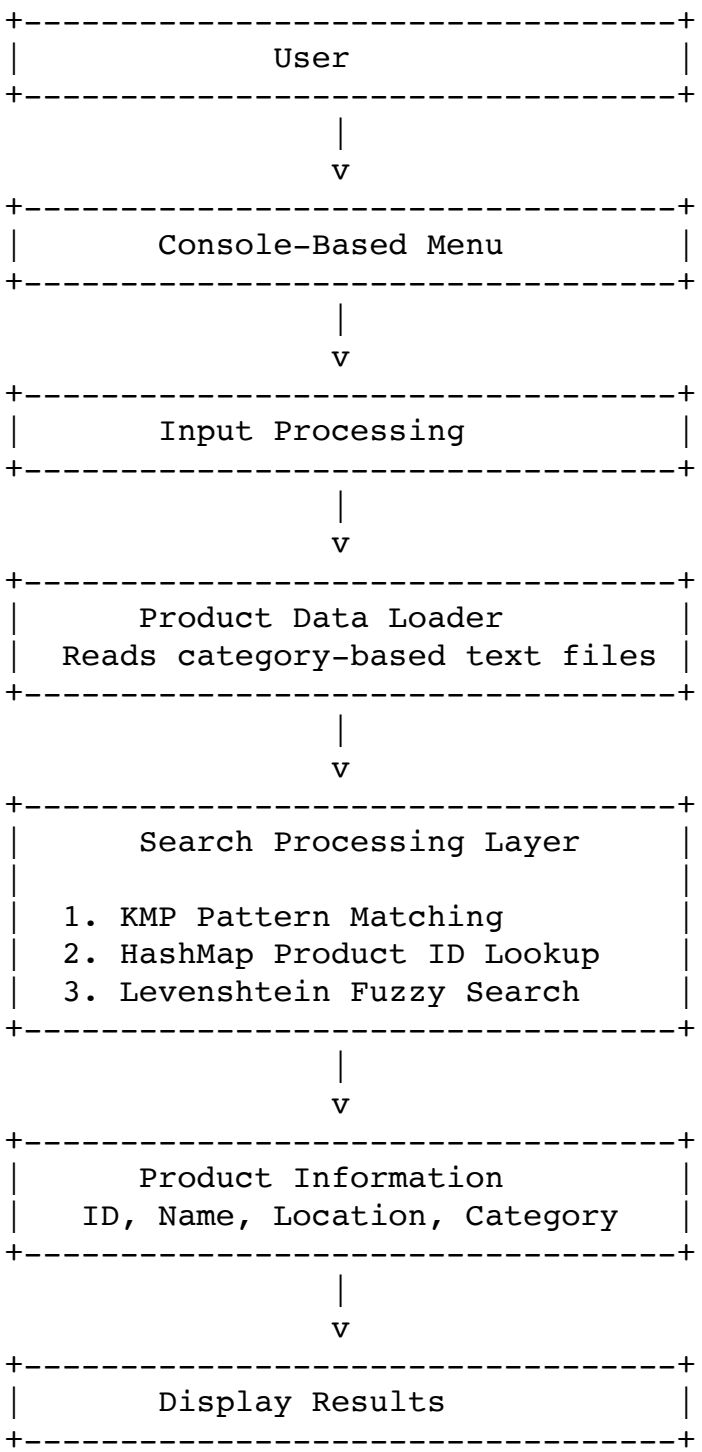


Fig. 3.1: Architecture of the Warehouse Inventory System – Product Locator

3.2 Description of Each Layer

3.2.1 Data Storage Layer

This layer contains the product information stored in category-based text files. Separating records by category makes the dataset easier to organize and maintain.

3.2.2 Data Loading Layer

The data loader reads the available files and converts the stored records into product objects or equivalent in-memory representations. This makes the information available to the searching modules.

3.2.3 Input Processing Layer

This layer accepts the user's menu selection and search input. It performs any necessary input normalization, such as handling letter-case differences.

3.2.4 Search Processing Layer

This layer contains the main algorithms used by the application:

- KMP for product-name pattern matching.
- HashMap for product-ID lookup.
- Levenshtein Distance for approximate product-name matching.

3.2.5 Result Presentation Layer

This layer displays the products matching the search criteria. If no suitable result is available, the application displays an appropriate not-found message.

3.3 Data Structures Used

3.3.1 Product Record

A product record represents an individual item in the warehouse dataset.

It contains the product ID, product name, storage location, and source file information.

Using a structured product representation keeps related fields together and makes it easier to process the same record through different searching techniques.

3.3.2 ArrayList or List-Based Collection

A list-based collection can hold the loaded product records and support sequential traversal.

It is useful when the program needs to examine multiple products during pattern matching or fuzzy searching.

For a dataset containing (n) products, traversing the complete collection takes (O(n)) time, excluding the cost of the operation performed on each product.

3.3.3 HashMap

A HashMap stores key-value pairs. In this project, the product ID can be used as the key and the corresponding product record as the value.

For example:

```
HashMap<String, Product> productMap;
```
A product can then be retrieved using its ID.

HashMap lookup takes expected (O(1)) time under normal hashing assumptions. In the worst case, performance can be less efficient because of collisions and implementation details.

3.3.4 Dynamic Programming Table

Levenshtein Distance uses a two-dimensional table to store the minimum edit distance between prefixes of two strings.

For strings of lengths (m) and (n), the standard dynamic programming algorithm requires (O(mn)) time and (O(mn)) space.

This avoids repeatedly solving the same smaller subproblems.

3.4 Algorithms Used

3.4.1 Knuth-Morris-Pratt Algorithm

The Knuth-Morris-Pratt algorithm is a string-searching algorithm used to find occurrences of a pattern within a text.

It preprocesses the search pattern to construct an LPS array.

LPS stands for Longest Proper Prefix which is also a Suffix. The array helps determine how far the pattern can shift after a mismatch without repeating unnecessary character comparisons.

In this project, KMP is used to search product names for a given pattern.

Main steps:

1. Accept the product-name pattern.
2. Normalize the input if required.
3. Construct the LPS array.
4. Compare the pattern against each product name.
5. Use the LPS values to avoid unnecessary comparisons after mismatches.
6. Display the matching products.

For a pattern of length (m) and a product-name string of length (n), KMP requires (O(m+n)) time for one string, including pattern preprocessing.

When the algorithm is applied to all product names, the total time depends on the total number of characters processed across those names.

3.4.2 HashMap-Based Search

HashMap is used to associate product IDs with their corresponding records.

The product ID acts as a key, and the product record acts as a value.

When the user enters a product ID, the system checks the map for the corresponding record and displays the product details if the key exists.

Main steps:

1. Load the product records.
2. Insert each valid product ID and its corresponding record into the map.
3. Accept the product ID from the user.
4. Search the map using the entered key.
5. Display the corresponding product or a not-found message.

Expected lookup time is (O(1)), assuming a suitable hash distribution and normal operating conditions.

3.4.3 Levenshtein Distance Algorithm

Levenshtein Distance measures how different two strings are by calculating the minimum number of single-character insertions, deletions, and substitutions required to transform one string into another.

For example, the strings `book` and `books` have a Levenshtein Distance of 1 because one insertion is sufficient to transform the first string into the second.

The algorithm uses dynamic programming.

Let (D[i][j]) represent the minimum edit distance between the first (i) characters of string (A) and the first (j) characters of string (B).

The initialization is:

[
D[i][0]=i
]

[
D[0][j]=j
]

If the current characters are equal, the substitution cost is zero. Otherwise, the substitution cost is one.

The recurrence is:

$$
D[i][j]=\min
\begin{cases}
D[i-1][j]+1\\
D[i][j-1]+1\\
D[i-1][j-1]+c
\end{cases}
$$

where (c=0) when the current characters match and (c=1) otherwise.

The final value (D[m][n]) gives the edit distance between the two complete strings.

In this project, the distance is used to identify product names that are sufficiently similar to the entered search term. The acceptance threshold depends on the implementation.

3.4.4 Comparison of Algorithms

Algorithm	Purpose	Typical Complexity
KMP	Product-name pattern matching	(O(m+n)) per string
HashMap	Product-ID lookup	Expected (O(1)) lookup
Levenshtein Distance	Approximate string matching	(O(mn)) per string pair

Here, (m) and (n) denote the relevant pattern and text lengths. For Levenshtein Distance, they represent the lengths of the two strings being compared.

3.5 Module Description

Module 1: Dataset Loader

Reads product records from the category files and makes the data available to the application.

Module 2: KMP Search

Searches product names for the entered pattern using the KMP algorithm and its LPS array.

Module 3: Product ID Search

Uses a HashMap to identify the product corresponding to an entered product ID.

Module 4: Fuzzy Search

Calculates edit distances between the user's search term and product names to identify possible matches.

Module 5: User Interface

Provides a menu through which the user selects the desired search operation and exits the application.

Module 6: Result Display

Displays matching product information or an appropriate message when no result is available.

3.6 Complexity Analysis

Let:

- (N) be the number of products.
- (L) be the average product-name length.
- (m) be the search-pattern length.

KMP Search: Searching every product name takes approximately (O(N(L+m))) time when all names are of comparable length and each name is processed separately.

HashMap Lookup: Searching for one product ID takes expected (O(1)) lookup time after the map has been constructed. Building the map from (N) records takes expected (O(N)) time.

Fuzzy Search: Comparing one query with every product name using standard Levenshtein Distance takes approximately (O(NLm)) time when the query length is (m) and average product-name length is (L).

The actual execution time depends on the number of products, their names, the implementation, and the computer running the application.

CHAPTER 4: IMPLEMENTATION

4.1 Implementation Overview

The Warehouse Inventory System – Product Locator is implemented using Java. The application follows a menu-driven approach and uses a predefined product dataset stored in text files.

The implementation is divided into data loading, product representation, search processing, and output presentation.

The system provides three main searching techniques: KMP-based pattern matching, HashMap-based product-ID lookup, and fuzzy matching using Levenshtein Distance.

This modular approach allows each algorithm to be understood and tested independently.

4.2 Product Data Representation

The product record contains the information required to identify and locate an item.

The main attributes are:

- Product ID
- Product name
- Storage location
- Source category file

These attributes allow the application to display the relevant product details after a successful search.

The source file information can also help identify the category from which the product record was loaded.

4.3 Dataset Loading

The application reads product information from the files stored in the `data` directory.

The loading procedure examines the available category files, reads their records, and converts valid entries into the application's internal product representation.

The loaded products can then be processed by the searching algorithms.

The dataset-loading module is important because the searching modules depend on correctly parsed product information.

The application should also handle unavailable files and malformed records appropriately.

4.4 KMP-Based Product Search

The KMP search module searches for a user-provided pattern within product names.

The algorithm first preprocesses the pattern to create the LPS array. It then compares the pattern against each product name.

When a mismatch occurs, the LPS array determines the next pattern position. This prevents the algorithm from restarting the comparison unnecessarily.

The search process continues until the text has been examined or a matching pattern has been found, according to the search implementation.

If matching product names are found, their details are displayed to the user.

4.5 HashMap-Based Product Identification

The HashMap module identifies products using their product IDs.

During data preparation, each valid product ID is associated with its product record.

When the user enters an ID, the application checks whether the key exists in the map. If it exists, the corresponding record is retrieved and displayed.

If the entered ID is not available, the application displays a not-found message.

This approach avoids the need to scan every product record for each ID lookup, assuming the map has already been constructed.

4.6 Fuzzy Search Using Levenshtein Distance

The fuzzy-search module is designed to handle search inputs that differ slightly from the actual product names.

For example, a user might enter an incorrect spelling or omit a character while searching for an item.

The algorithm calculates the edit distance between the search term and candidate product names. Smaller distances indicate greater similarity.

The system uses the calculated distance to determine whether a product name should be considered a possible match.

This feature improves the flexibility of name-based searching, although it may return multiple possible matches or miss a product if the configured threshold is too restrictive.

4.7 Main Menu and Control Flow

The application presents a menu containing the available search operations.

The general menu structure is:

1. Search by product name using KMP.
2. Search by product ID using HashMap.
3. Search by product name using fuzzy matching.
4. Exit the application.

The program accepts the user's choice and calls the appropriate search operation.

After displaying the results, the application allows the user to perform another search or exit.

Invalid menu choices are handled by displaying an appropriate message.

4.8 Pseudocode

Algorithm 1: Load Product Dataset

```
ALGORITHM LoadProducts

BEGIN
    Initialize an empty product collection

    FOR each category file in the data directory
        Open the category file

        FOR each record in the file
            Read product ID
```

```
            Read product name
            Read product location
            Read source file information

            IF the record is valid
                Create a product record
                Add the product to the collection
            END IF
        END FOR

        Close the category file
    END FOR

    RETURN product collection
END
```

Algorithm 2: KMP Product Search

```
ALGORITHM KMPSearchProducts(products, pattern)

BEGIN
    Normalize the search pattern
    Construct the LPS array for the pattern

    FOR each product in products
        Normalize the product name
        Search for the pattern using KMP

        IF the pattern is found
            Display product ID
            Display product name
            Display product location
        END IF
    END FOR

    IF no matching product is found
        Display "Product not found"
    END IF
END
```

Algorithm 3: HashMap Product ID Search

```
ALGORITHM SearchByProductID(productMap, productID)

BEGIN
    IF productMap contains productID
        Retrieve the corresponding product

        Display product ID
        Display product name
        Display product location
    ELSE
        Display "Product not found"
```

```
    END IF
END
```

Algorithm 4: Levenshtein Distance

```
ALGORITHM LevenshteinDistance(A, B)

BEGIN
    m = length(A)
    n = length(B)

    Create a table D of size (m + 1) x (n + 1)

    FOR i = 0 TO m
        D[i][0] = i
    END FOR

    FOR j = 0 TO n
        D[0][j] = j
    END FOR

    FOR i = 1 TO m
        FOR j = 1 TO n

            IF A[i - 1] equals B[j - 1]
                cost = 0
            ELSE
                cost = 1
            END IF

            D[i][j] = minimum of:
                D[i - 1][j] + 1
                D[i][j - 1] + 1
                D[i - 1][j - 1] + cost

        END FOR
    END FOR

    RETURN D[m][n]
END
```

Algorithm 5: Fuzzy Product Search

```
ALGORITHM FuzzySearch(products, query, threshold)

BEGIN
    Normalize the query
    found = FALSE

    FOR each product in products
        Calculate Levenshtein Distance between
        query and product name
```

```
        IF distance satisfies the matching threshold
            Display product details
            found = TRUE
        END IF
    END FOR

    IF found is FALSE
        Display "No similar product found"
    END IF
END
```

Algorithm 6: Main Program

```
ALGORITHM Main

BEGIN
    Load the product dataset
    Prepare the required search data structures

    REPEAT
        Display the main menu
        Read the user's choice

        CASE choice OF

            1:
                Read product-name pattern
                Perform KMP product search

            2:
                Read product ID
                Perform HashMap lookup

            3:
                Read product-name query
                Perform fuzzy product search

            4:
                Exit the application

            OTHERWISE:
                Display "Invalid choice"

        END CASE

    UNTIL choice equals 4
END
```

CHAPTER 5: RESULTS AND DISCUSSION

5.1 System Output

The application provides a menu-driven interface that allows users to select the required searching method.

The available operations are KMP-based name searching, product-ID lookup through a HashMap, and fuzzy name searching using Levenshtein Distance.

After the user selects an operation and enters a search term, the system processes the request and displays the matching product information when available.

The expected output includes product IDs, product names, and storage locations based on the fields present in the dataset.

5.2 Functional Testing

The following test cases can be used to evaluate the system. Record the actual results after executing each test.

Test Case	Input or Action	Expected Result
TC1	Start the application	Main menu is displayed
TC2	Enter a valid product-name pattern	Matching products are displayed
TC3	Enter a valid product ID	Corresponding product details are displayed
TC4	Enter an invalid product ID	Product-not-found message is displayed
TC5	Enter a slightly misspelled product name	A similar match is displayed if it meets the fuzzy-search threshold
TC6	Enter a pattern with no matching product	No-match message is displayed
TC7	Enter an invalid menu option	Invalid-choice message is displayed
TC8	Select the exit option	Application terminates normally

Note: The expected results above describe the intended behavior. The actual result column should be completed from real test executions before submission.

5.3 Algorithm Analysis

The three algorithms are evaluated according to their purpose and computational complexity.

KMP Algorithm: KMP performs pattern matching in linear time relative to the combined lengths of the pattern and text being searched. Its LPS array helps reduce unnecessary comparisons.

HashMap: HashMap supports efficient product-ID lookup, with expected constant-time access under normal hashing assumptions. The initial map construction requires processing the product records.

Levenshtein Distance: The standard dynamic programming method takes (O(mn)) time and (O(mn)) space to compare two strings of lengths (m) and (n). Applying it to all product names increases the total work with the size of the dataset.

These results show that different algorithms are appropriate for different searching requirements. HashMap is suitable for identifier lookup, KMP is suitable for substring matching, and Levenshtein Distance is suitable for approximate text comparison.

5.4 Result Discussion

The project demonstrates how combining multiple searching techniques can provide a more flexible product locator than relying on a single exact-match method.

KMP enables the application to search for patterns within product names. HashMap provides an efficient way to retrieve a record using a product ID. Levenshtein Distance provides a mechanism for finding similar names when the input contains minor spelling differences.

The modular design also allows the search methods to be tested and improved independently.

The effectiveness of the fuzzy-search module depends on the configured threshold and the characteristics of the product names. A very strict threshold may reject useful matches, while a permissive threshold may produce irrelevant results.

Similarly, actual execution performance depends on the dataset size, string lengths, data-loading process, and implementation details.

Therefore, the project illustrates the algorithmic advantages of the selected methods, while quantitative performance claims should be based on measurements collected during testing.

CHAPTER 6: CHALLENGES AND LIMITATIONS

6.1 Challenges Faced During Development

6.1.1 Organizing Product Data

One challenge was organizing product information stored in multiple category-based text files. The records had to be read and represented consistently so that each searching method could access the required information.

6.1.2 Implementing Pattern Matching

Implementing KMP required understanding the construction of the LPS array and its role in handling mismatches. Incorrect LPS calculations can lead to missed matches or incorrect search behavior.

6.1.3 Efficient Product Identification

The application needed a way to identify a product directly from its ID. A HashMap was selected to associate each product ID with the corresponding product record.

6.1.4 Handling Spelling Variations

Exact pattern matching cannot identify every intended product when the user enters an incorrect spelling. Implementing Levenshtein Distance introduced a way to compare strings based on the number of character edits required to transform one into another.

6.1.5 Input and Error Handling

The application also needs to handle invalid menu choices, unavailable product IDs, unsuccessful searches, and potential data-file problems.

6.2 Solutions Applied

The project addresses these challenges through the following approaches:

- Product information is stored in category-based files and represented using structured records.
- KMP uses an LPS array to avoid repeated comparisons during pattern matching.
- HashMap associates product IDs with their corresponding records for efficient lookup.
- Levenshtein Distance uses dynamic programming to calculate the similarity between strings.
- The menu-driven structure separates the available search operations.
- Not-found messages and input validation help make the application easier to use.

These approaches demonstrate how appropriate algorithm and data-structure choices can address different requirements within the same application.

6.3 Limitations of the System

The current project has several limitations.

Static Dataset: The product records are loaded from predefined files. The application does not necessarily support live updates from an external inventory database.

Limited Inventory Operations: The main focus is searching and locating products. Advanced functions such as adding products, updating stock quantities, purchasing, and billing are outside the defined core functionality.

Console Interface: The application uses a menu-driven console interface rather than a graphical user interface.

Fuzzy Matching Threshold: Approximate searching depends on the selected edit-distance threshold. The threshold may require adjustment for different product names and datasets.

Memory Usage: Loading product records into memory and maintaining additional search structures consumes memory. Resource requirements increase with the size of the dataset.

No Real-Time Warehouse Integration: The system does not automatically connect to warehouse sensors, barcode scanners, or live stock-tracking services unless such functionality is added separately.

Limited Performance Evaluation: Reliable comparisons of actual execution time and memory consumption require systematic testing on a defined dataset.

CHAPTER 7: CONCLUSION AND FUTURE WORK

7.1 Conclusion

The project **"Warehouse Inventory System – Product Locator"** demonstrates the practical application of Data Structures and Algorithms to a warehouse product-searching problem.

The system is developed using Java and uses category-based text files to store product information. It provides a menu-driven interface through which users can search for products using product-name patterns, product IDs, and approximate text matching.

The Knuth-Morris-Pratt algorithm supports efficient pattern matching by using the LPS array to reduce unnecessary character comparisons. HashMap provides an efficient mechanism for identifying products through their IDs. Levenshtein Distance uses dynamic programming to calculate the minimum number of character edits required to transform one string into another, supporting fuzzy product searching.

The project demonstrates that different search operations require different algorithmic approaches. Selecting an appropriate data structure or algorithm can improve the efficiency and flexibility of an application.

Overall, the project provides practical experience in Java programming, file handling, string processing, algorithm implementation, and complexity analysis. It also establishes a foundation for developing more advanced warehouse inventory and product-management applications.

7.2 Future Enhancements

The system can be improved in several ways.

7.2.1 Database Integration

A database such as MySQL could be introduced to store product records persistently and support more advanced inventory operations.

7.2.2 Graphical User Interface

A graphical interface could replace the console menu to make searching more convenient and accessible.

7.2.3 Inventory Management

Additional modules could support adding products, updating quantities, deleting records, and tracking stock availability.

7.2.4 Barcode and QR Code Support

Barcode or QR code scanning could be incorporated to identify products quickly and reduce manual data entry.

7.2.5 Advanced Search and Filtering

The system could support category-based filters, location filters, prefix searching, and ranking of fuzzy matches according to their edit distances.

7.2.6 Performance Optimization

Performance could be improved through efficient indexing, optimized string comparison, and memory-efficient implementations of the searching algorithms.

7.2.7 Automated Testing

A larger collection of test cases could be used to evaluate correctness, execution time, memory usage, and behavior under different input conditions.

7.2.8 Real-Time Inventory Integration

Future versions could integrate with warehouse management systems to retrieve updated product locations and stock quantities.

7.3 Skills Learned During Development

The development of this project helped strengthen the following skills:

- Understanding and implementing fundamental data structures.
- Applying the KMP string-matching algorithm.
- Using HashMap for key-based retrieval.
- Implementing Levenshtein Distance using dynamic programming.
- Working with Java collections and text files.
- Designing a menu-driven program.
- Dividing an application into logical modules.
- Understanding time and space complexity.

- Testing search operations and handling unsuccessful inputs.
- Applying algorithmic concepts to a practical problem.

The project helped connect theoretical concepts from the Data Structures and Algorithms course with a practical software application.

REFERENCES

[1] Oracle, "Java Documentation," Oracle. Available: https://docs.oracle.com/en/java/

[2] Oracle, "Java Collections Framework," Oracle. Available: https://docs.oracle.com/javase/8/docs/technotes/guides/collections/overview.html

[3] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA: MIT Press, 2022.

[4] D. E. Knuth, J. H. Morris Jr., and V. R. Pratt, "Fast Pattern Matching in Strings," *SIAM Journal on Computing*, vol. 6, no. 2, pp. 323–350, 1977.

[5] V. I. Levenshtein, "Binary Codes Capable of Correcting Deletions, Insertions, and Reversals," *Soviet Physics Doklady*, vol. 10, pp. 707–710, 1966.

[6] R. Sedgewick and K. Wayne, *Algorithms*, 4th ed. Addison-Wesley, 2011.

[7] Java Platform Documentation, "HashMap Class," Oracle. Available: https://docs.oracle.com/en/java/javase/

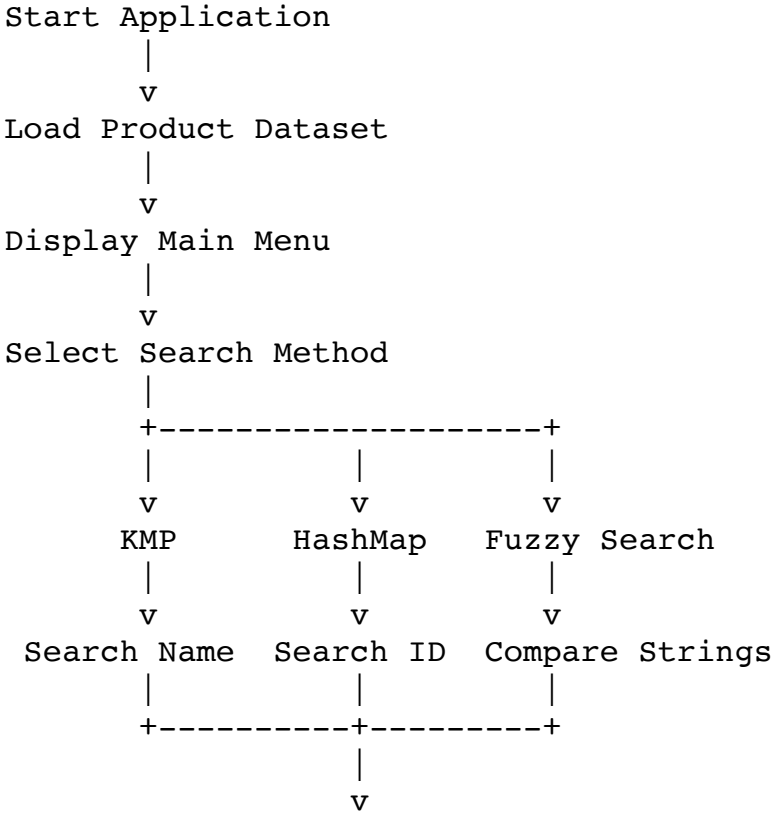
APPENDIX A: PROJECT DIRECTORY STRUCTURE

The project can be organized as follows:

```
WarehouseInventorySystem/
|
|-- src/
|    |-- Main.java
|    |-- Product.java
|    |-- KMP.java
|    |-- FuzzySearch.java
|
|-- data/
|    |-- category files (.txt)
|
|-- docs/
|    |-- project documentation
|
|-- results/
|    |-- test outputs and screenshots
|
|-- reports/
|    |-- project reports
|
|-- README.md
```

The exact source filenames may differ according to the final implementation.

APPENDIX B: SAMPLE SEARCH WORKFLOW



```
Display Results
        |
        v
Continue or Exit
```

Fig. B.1: Workflow of the Warehouse Inventory System – Product Locator

APPENDIX C: SAMPLE TESTING RECORD

Use the following table to record actual results during execution.

Test ID	Search Method	Input	Actual Result	Status
T1	KMP	[Valid product-name pattern]	[Record observed output]	[Pass/ Fail]
T2	HashMap	[Valid product ID]	[Record observed output]	[Pass/ Fail]
T3	HashMap	[Invalid product ID]	[Record observed output]	[Pass/ Fail]
T4	Fuzzy Search	[Slightly misspelled product name]	[Record observed output]	[Pass/ Fail]
T5	Menu	Invalid menu choice	[Record observed output]	[Pass/ Fail]

Complete this appendix after running the program so that the submitted report accurately reflects the implementation and its observed behavior.